

KV Database

基于 Rust 的教学型关系数据库设计与实现

成员 1: 【姓名、学号、分工】

成员 2: 【姓名、学号、分工】

成员 3: 【如无请删除】

【提交日期】

摘要

本文介绍使用 Rust 2024 实现的教学型关系数据库 KV Database。项目从固定大小磁盘页面、superblock、空闲页链表和 B+Tree 开始，贯通 catalog、缓冲池、SQL 处理、事务管理、MVCC、表级锁、MySQL Wire Protocol、HTTP API 与 React 工作台。当前基线包含 74 个 Rust 测试和 87 项协议/持久化测试。项目参考 SQLite、PostgreSQL、MySQL InnoDB 和 CMU BusTub 的公开设计思想，但没有复制其源代码。

关键词: Rust; 关系数据库; B+Tree; 事务; MVCC; 持久化

目录

1	项目名称与团队分工	3
1.1	项目名称	3
1.2	团队成员与分工	3
2	项目背景与目标	3
2.1	项目背景	3
2.2	项目目标	3
2.3	功能范围	4
3	系统设计与实现思路	4
3.1	总体架构	4
3.2	数据库文件和 superblock	4
3.3	Slotted Page	5
3.4	B+Tree 和缓冲池	5

3.5 SQL、事务与接口	5
4 关键实现代码索引	6
5 实验环境与复现	6
5.1 启动命令	6
5.2 质量门禁	6
6 实验结果	7
6.1 功能结果	7
6.2 自动化结果	7
6.3 截图占位	8
7 开源项目参考与差异	9
7.1 SQLite	9
7.2 PostgreSQL	9
7.3 MySQL InnoDB	9
7.4 CMU BusTub	9
8 代码规范与工程质量	10
9 局限性与后续工作	10
10 结论	11
A 复现实验 SQL	11
B 提交前替换清单	11

1 项目名称与团队分工

1.1 项目名称

KV Database: 基于 Rust 的教学型关系数据库。

1.2 团队成员与分工

提交前将占位符替换为真实信息，并确保与代码提交和视频一致。

表 1: 团队成员与主要分工

姓名	学号	主要分工
【姓名 1】	【学号 1】	【存储、事务或测试】
【姓名 2】	【学号 2】	【SQL、协议或服务端】
【姓名 3】	【学号 3】	【Web、工程化或文档】

2 项目背景与目标

2.1 项目背景

数据库系统同时涉及数据结构、操作系统、网络、编程语言和并发控制。仅调用现成数据库接口无法观察 SQL 如何变成页面读写，也无法验证事务和持久化的具体行为。因此，本项目实现一个规模可控但具有完整请求路径的教学型关系数据库。

2.2 项目目标

1. 实现固定 4096 字节页面、superblock、catalog 和可复用空闲页链表。
2. 实现固定阶 B+Tree，支持查找、插入、删除、分裂和叶节点范围扫描。
3. 实现 SQL Lexer、Parser、Planner、Executor，覆盖 DDL、DML、条件查询、排序和等值 JOIN。
4. 实现事务生命周期、写缓冲、事务内读己之写、MVCC 可见性和表级锁。
5. 实现 MySQL Wire Protocol 最小子集、本地 HTTP API 和 React 工作台。
6. 用自动化测试验证协议边界、页面损坏、缓存失效和重启持久化。

2.3 功能范围

表 2: 当前版本功能范围

模块	已实现内容
SQL	CREATE/DROP TABLE、CREATE INDEX、SELECT、INSERT、UPDATE、DELETE
查询	投影、WHERE、比较与逻辑表达式、ORDER BY、等值 JOIN
事务	BEGIN、COMMIT、ROLLBACK、写集、MVCC、表级锁
存储	4 KiB 页面、Pager、slotted page、缓冲池、B+Tree、catalog、空闲页复用
接口	MySQL TCP、/api/state、/api/query、/api/reset

3 系统设计与实现思路

3.1 总体架构

```

MySQL client --> kv-network --+
                                +--> kv-sql --> kv-txn --> kv-storage --> kv.db
React UI -----> demo HTTP API -+

```

1. kv-common: 共享类型、错误和 Pager trait。
2. kv-network: 握手、命令包、分片重组和结果集编码。
3. kv-sql: Lexer、Parser、Planner 和 Executor。
4. kv-txn: 事务 ID、读写集、版本可见性和表级锁。
5. kv-storage: 页面、Pager、缓冲池、B+Tree、catalog 和刷盘。
6. kv-server: MySQL TCP 服务和本地 HTTP 演示 API。

模块依赖保持单向：网络和 HTTP 调用 SQL 执行器，执行器依赖事务与存储，存储层不依赖前端。

3.2 数据库文件和 superblock

数据库文件由固定 4096 字节页面组成，page 0 为 superblock。

表 3: superblock 字段

偏移	字段	含义
0-7	<code>next_page_id</code>	下一个未分配页号
8-15	<code>free_list_head</code>	空闲页链表头
16-23	<code>catalog_root</code>	catalog B+Tree 根页
24-31	<code>magic</code>	KVDBPAGE
32-35	<code>version</code>	当前为 1

`DiskPager::open` 检查文件长度、魔数、版本、页号范围和空闲链表环。空闲页前 8 字节保存前一个链表头，重新分配时从链表头弹出。该结构适合教学，但不包含生产数据库的 WAL、vacuum 和复杂恢复。

3.3 Slotted Page

`crates/kv-storage/src/page.rs` 使用槽目录和 tuple 数据分离的布局。页头保存 page id、tuple 数量、free start、free end 和标志位；槽项保存 tuple 偏移和长度。目录向前增长，tuple 从页尾向前增长。`SlottedPage::validate_layout` 检查槽目录、tuple 偏移、页面边界和重叠关系，将损坏数据转换为错误而不是触发越界访问。

3.4 B+Tree 和缓冲池

B+Tree 内部节点保存分隔键和 child page，叶节点保存有序键值并通过 `next` 连接。插入时向下查找叶节点，按序插入；超容量后分裂节点、向父节点上推分隔键，根分裂时创建新根。固定阶设计在超出 4 KiB 时返回错误，尚未实现完整删除再平衡。

`Pager trait` 抽象页面读写、分配、释放、刷盘和 catalog 根页。`BufferedPager` 使用 `Mutex` 保护的 LRU 缓存。释放页面时显式删除缓存，避免页号复用后读取旧副本；该行为由 `freed_page_is_removed_from_cache` 覆盖。

3.5 SQL、事务与接口

SQL 依次经过 Lexer、Parser、Planner 和 Executor。Executor 从 catalog 和 B+Tree 读取数据，并结合事务上下文执行计划。事务写操作进入写缓冲和写集，提交时应用，回滚时丢弃；MVCC 判断版本可见性，表级锁限制冲突写入。项目不宣称完整 ACID 崩溃恢复，因为没有 WAL。

MySQL 服务实现握手、命令包、OK/ERR 响应和结果集编码，支持 TCP 分片重组并拒绝超过 8 MiB 或截断的包。HTTP API 提供 `GET /api/state`、`POST /api/query` 和 `POST /api/reset`，请求体限制 64 KiB。React 工作台只消费真实状态快照，不复制 SQL 规则，展示 SQL、执行阶段、结果、schema、索引数量和后端 `durationMicros`。

4 关键实现代码索引

表 4: 代码位置与说明

代码位置	说明
crates/kv-storage/src/pager.rs	文件格式、superblock、空闲页链表和刷盘
crates/kv-storage/src/page.rs	PageHeader、SlotEntry、页面编解码和边界验证
crates/kv-storage/src/btree.rs	内部/叶节点、插入、分裂、删除和扫描
crates/kv-storage/src/buffer.rs	BufferPool、BufferedPager、LRU 和缓存失效
crates/kv-txn/src/manager.rs	事务生命周期、写集和提交/回滚
crates/kv-txn/src/mvcc.rs	版本链和可见性判断
crates/kv-sql/src/lexer.rs	Token 定义和词法扫描
crates/kv-sql/src/parser.rs	SQL 子集语法分析
crates/kv-sql/src/planner.rs	AST 到执行计划
crates/kv-sql/src/executor.rs	查询、写入、事务和 JOIN
crates/kv-network/src/protocol.rs	MySQL 包解析和结果编码
crates/kv-server/src/demo_http.rs	HTTP 路由、JSON 和请求大小限制
demo-client/src/App.jsx	工作台状态、执行历史和展示逻辑

5 实验环境与复现

5.1 启动命令

```
cargo run -p kv-server
cd demo-client
npm ci
npm run dev
```

默认 MySQL 地址为 127.0.0.1:3307, HTTP API 为 127.0.0.1:8080, 前端为 127.0.0.1:5173, 数据文件为 kv_data/kv.db。独立测试可设置 KV_DATA_DIR=target/video-demo。

5.2 质量门禁

```
cargo fmt --check --all
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace --all-targets
cargo doc --workspace --no-deps
cd demo-client && npm ci && npm run build && cd ..
python test_protocol.py
git diff --check
```

```
bash scripts/run-and-show-tests.sh
```

test_protocol.py 自行管理 3307 端口的测试服务并验证协议、SQL、事务、错误处理和重启持久化，因此运行完整模式前应停止演示服务。

6 实验结果

6.1 功能结果

表 5: 主要功能验证结果

实验项	操作与结果	状态
建表与 catalog	创建 users、orders；状态接口返回列定义；重启后表结构可加载	通过
批量插入	users 3 行、orders 3 行，工作台显示 6 条记录	通过
二级索引	创建 idx_users_name，schema 显示索引数量	通过
条件、排序和 JOIN	WHERE、ORDER BY 和 users.id=orders.uid 返回正确结果	通过
事务读己之写	Ada 年龄更新为 99 后同一事务内可读到 99	通过
回滚	ROLLBACK 后 Ada 年龄恢复为 28	通过
持久化	关闭并重新打开数据库后，catalog 和记录仍可查询	通过
边界处理	分片、截断、超大包、HTTP 超限、非法 JSON 和损坏页面返回错误	通过

6.2 自动化结果

表 6: 当前测试基线

检查	结果
Rust 格式	cargo fmt --check --all 通过
Clippy	-D warnings 通过，无警告
Rust workspace	74 个测试通过
前端构建	Vite production build 通过
协议/持久化	87 项通过，含重启恢复
文档生成	cargo doc --workspace --no-deps 通过

6.3 截图占位

提交前请用真实截图替换下列占位框，不能使用静态 mock。

【截图 1：SQL A、两张表、六条记录和索引】

图 1: 建表、插入和索引

【截图 2：JOIN 结果及 durationMicros】

图 2: 查询和后端耗时

【截图 3：99 与 ROLLBACK 后的 28】

图 3: 事务和回滚

【截图 4：测试汇总和 kv.db superblock】

图 4: 自动化测试和持久化文件

7 开源项目参考与差异

7.1 SQLite

参考 SQLite version-3.50.4 的 `src/btree.c` 和 Database File Format 文档, SQLite 以公有领域声明发布。对照 `freePage2` 的页号检查和 `free-list` 更新逻辑。SQLite 使用 `page 1` 的 `trunk/leaf freelist`; 本项目在 `pager.rs` 中以 `superblock` 加每个空闲页前 8 字节组成单链表。共同点是拒绝非法页号并维护可复用页面; 本项目增加文件长度、魔数、版本和 `freelist` 环检查, 并用重启测试验证复用。SQLite 的 `journal/WAL`、`vacuum` 和复杂事务恢复未实现。

7.2 PostgreSQL

参考 PostgreSQL REL_18_0 的 `src/include/storage/bufpage.h`, 使用 PostgreSQL License。其 `PageHeaderData` 包含 `pd_lower`、`pd_upper`、`pd_special` 和 `pd_linp`。本项目的 `PageHeader` 使用 `free_start/free_end`, `SlotEntry` 保存 `tuple` 范围, `SlottedPage::validate_layout` 负责边界检查。两者均分离槽目录和 `tuple`; 本项目省略 `LSN`、`checksum`、`special space` 和 `MVCC` 页面头。

7.3 MySQL InnoDB

参考 MySQL mysql-8.4.0 的 `storage/innobase/include/page0types.h`, 代码采用 GPLv2。对照 `PAGE_N_DIR_SLOTS`、`PAGE_N_RECS`、`PAGE_LEVEL` 和 `PAGE_INDEX_ID`。本项目在 `btree.rs` 中使用 `FLAG_INTERNAL`、`FLAG_LEAF`、键数量和叶节点 `next` 实现固定阶 B+Tree。InnoDB 的页面目录、聚簇索引、并发控制和恢复能力远超本项目; 本项目只保留教学闭环, 并在节点过页或字段损坏时返回错误。

7.4 CMU BusTub

参考 BusTub commit `f0d9e3753482d45f2b5919da1873684600b48508`, 采用 MIT License。其 `BufferPoolManager` 提供 `NewPage`、`DeletePage`、`CheckedReadPage`、`CheckedWritePage` 和 `FlushPage`。本项目在 `kv-common` 定义 `Pager` trait, 在 `buffer.rs` 中实现 `BufferPool` 和 `BufferedPager`。BusTub 使用 `page guard`、`pin count`、读写锁和 `replacer`; 本项目使用 Rust trait、`Mutex` 和 `LRU`, 并测试释放页面后的缓存失效。

表 7: 对比结论

维度	参考系统	本项目取舍
页面	SQLite 固定页；PostgreSQL slot- ted page	4 KiB 页面、superblock、槽页和严 格边界检查
空闲页	SQLite trunk/leaf freelist	页内单链表，跨重启复用并检测链 表环
索引	InnoDB 聚簇/二级索引和恢复	固定阶 B+Tree，叶链支持扫描，超 页返回错误
缓存	BusTub guard/pin/replacer	Pager trait、Mutex 和 LRU，释放 页时失效缓存
事务	成熟数据库 WAL、锁和磁盘 MVCC	写缓冲、内存版本链、表级锁，明 确不承诺崩溃原子恢复

以上引用均为结构思想和短代码对照，不是上游源码复制。报告和视频应保留官方链接、固定版本和许可证。

8 代码规范与工程质量

项目采用 Rust 2024 workspace 和最低 Rust 1.94。核心模块提供模块级文档及关键算法注释，错误通过 KvError 传播，禁止 unsafe、todo!、unimplemented! 和调试宏。CI 检查格式、Clippy、Rust 测试、前端构建和协议测试。

输入边界方面，MySQL 包限制为 8 MiB，HTTP 请求体限制为 64 KiB；页面解码校验长度和偏移；前端错误显示使用文本节点。这些措施服务于课程项目的正确性和可观察性，不等同于生产安全模型。

9 局限性与后续工作

- 没有 WAL、checkpoint 和完整崩溃恢复。
- B+Tree 尚未实现按字节利用率分裂、删除合并和完整再平衡。
- 二级索引的非唯一键、增量维护和重启恢复能力有限。
- MVCC 版本主要位于内存，没有完整磁盘版本链和垃圾回收。
- 锁粒度为表级，没有等待队列和完整死锁图检测。
- MySQL 协议只覆盖项目 SQL 子集。
- HTTP 演示接口共享单个会话，不适合公网或多租户。

后续优先级为：带 LSN 的 WAL 和 checkpoint；B+Tree 删除再平衡；持久化二级索引；MVCC 版本落盘与 vacuum；行级锁和等待图。

10 结论

本项目完成了从 SQL 输入到磁盘页面的 Rust 数据库最小闭环。系统可通过 MySQL 客户端和 Web 工作台接受请求，经过 Lexer、Parser、Planner、Executor、事务层和 B+Tree 存储返回结果；数据库文件可在重启后恢复 catalog 和记录。自动化测试验证了协议边界、页面损坏、缓存失效和持久化行为。

项目的课程价值在于每个层次均可观察并由小组自行实现。与 SQLite、PostgreSQL、InnoDB 和 BusTub 的关系是结构思想对照，不是代码复制。最终提交前仍需补齐真实成员信息、截图、GitHub 地址和视频文件，并再次运行质量门禁。

A 复现实验 SQL

```
CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100), age INT);
INSERT INTO users VALUES (1, 'Ada', 28), (2, 'Grace', 31), (3, 'Linus', 25);
CREATE INDEX idx_users_name ON users (name);
CREATE TABLE orders (id INT PRIMARY KEY, uid INT, amount FLOAT);
INSERT INTO orders VALUES (101, 1, 299.0), (102, 2, 499.0), (103, 1, 129.0);
SELECT * FROM users JOIN orders ON users.id = orders.uid;
BEGIN;
UPDATE users SET age = 99 WHERE id = 1;
SELECT * FROM users WHERE id = 1;
ROLLBACK;
SELECT * FROM users WHERE id = 1;
```

B 提交前替换清单

- 替换所有姓名、学号、分工、日期和仓库地址占位符。
- 用真实截图替换四个占位框。
- 根据最终测试结果更新数字。
- 使用 XeLaTeX 编译并检查表格、代码和目录没有溢出。
- 只提交 PDF，不用 Word 替代 PDF。